BCC1344 Programming Data Analysis

Final Exam Prediction Paper — Structured, Short-Answer & Coding Questions with Model Answers

Based on: Ch1 Introduction to Data Science · Ch2 Python Programming (Colab) · Ch3 Python Libraries · Ch5 NumPy & Data Transformation · Lab Assessment 1 (Pandas & Data Preparation). All code outputs in this paper were executed and checked (Python 3, NumPy 2.x, pandas).

0. How to Use This Paper

Topic	Likelihood	What they usually ask
Ch1 Data preparation (Collection / Wrangling / Management)	**Very high**	Define + 2 activities + example for each step
Ch1 Four types of analysis	**Very high**	Define each, write one question per type for a scenario
Ch1 Anatomy of data (variable types, data types, metadata)	**Very high**	Classify columns of a given table
Ch2 if/elif/else, loops, list, dict	**Very high**	Trace output, write short program
Ch2 Class / object / method / self	High	Define terms, write a small class
Ch3 NumPy vs Pandas vs Matplotlib	High	Purpose, import statement, how they work together
Ch5 NumPy arrays (create, attributes, indexing, vectorised maths, aggregation)	**Very high**	Predict output, write code
Ch5 Data transformation in pandas (new column, rename, filter, sort, apply, groupby, get_dummies, pd.cut, normalise)	**Very high**	Write one-liners for a given DataFrame
Lab 1 workflow (load → inspect → clean → describe → groupby → reflect)	**Very high**	Full coding question + interpretation + reflection
Ch1 STAR framework, population vs sample, storytelling	Medium	Short answer, 3–4 marks

Exam technique. Structured answers score by points, so write one point per line: *term → meaning → example*. For coding questions, always write the `import` line, use the exact variable name in the question (e.g. `df`), and add a short comment per step — markers give method marks even if one line is wrong. Note: Chapter 4 (Pandas) slides were not in the material given; pandas questions here follow Lab 1 and the pandas code shown in Chapter 5.

SECTION A — Chapter 1: Introduction to Data Science

A1. (a) Differentiate between *data* and *data science*. (b) Data science is formed by combining three fields. Name and explain each field using a student performance analysis example. **[2 + 6 = 8]**

MODEL ANSWER

(a) **Data** is raw facts and figures (numbers, text, records) collected from a source, e.g. students' marks. **Data science** is the process of using statistics, programming and domain knowledge to turn that data into insights that answer a question or support a decision. Big data = a large combination of data from many sources.

(b)

1. **Statistics** – to understand the data. E.g. calculate the average mark, pass rate and performance trend.
2. **Computer Science** – to process the data using code/tools. E.g. write Python code or build a Power BI dashboard of results.
3. **Domain Knowledge** – to understand the real problem. E.g. know what counts as a good or bad result in education (e.g. below 50 = fail, CGPA 3.5 = dean's list).

Combining all three = doing data science.

A2. Explain the difference between a *population* and a *sample*. Give one example of each and state one limitation of using a sample. **[5]**

MODEL ANSWER

Population: the entire group of interest. Example: all used-car owners in the United States.
Sample: a smaller subset chosen to represent the population. Example: used-car owners in Chattanooga, Tennessee.
We use samples because collecting data from the whole population is costly and slow.
Limitation: a sample will not be 100% accurate; if it is not representative (e.g. one city only), results can be biased and may not generalise to the population.

A3. Describe the three main stages of the data science lifecycle. **[6]**

MODEL ANSWER

The lifecycle starts with a **question/problem** to investigate (what are we trying to solve?), then:

1. **Data Preparation** – collecting, cleaning, transforming, integrating and managing data so it is structured for analysis, storytelling and decision making.
2. **Data Analysis** – examining the prepared data to uncover valuable insights and guide decisions (descriptive, diagnostic, predictive, prescriptive).
3. **Data Storytelling** – communicating the insights through summaries, visualisations and narratives so the audience can understand and act.

A4. Refer to the table below.

student_id	name	age	gpa	favourite_pet	voted
S001	Ali	20	3.25	Cat	True
S002	Siti	21	3.80	Dog	False

(a) Identify the observational unit, one variable and one value. (b) Classify each of *age, gpa, favourite_pet, voted* as quantitative or categorical, and state its data type. (c) Define metadata and give three examples for this table. **[3 + 4 + 4 = 11]**

MODEL ANSWER

(a) Observational unit = **one student** (each row). Variable = e.g. **gpa** (a recorded characteristic, a column). Value = e.g. **3.25** (the attribute given to one student).
(b)

Column	Variable type	Data type	Reason
age	Quantitative	Integer	Numeric, maths makes sense, no decimal
gpa	Quantitative	Float	Numeric with decimal point
favourite_pet	Categorical	Text (string)	Category label, no maths
voted	Categorical	Boolean	Only two possible values True/False

(c) **Metadata** is data about data – information that describes and gives context to a dataset. Examples: variable names (`student_id, gpa…`), units of measurement (age in years, GPA on 4.0 scale), creation date of the file, number of missing values per column.

A5. Data preparation consists of three major steps. For each step, define it, list two activities and give one example. **[9]**

MODEL ANSWER

Step	Definition	Activities	Example
1. Data Collection	Gathering raw data from various sources	Import Excel/CSV files; extract from databases, Google Forms/surveys, websites, sensors/logs	Collect students' marks from Excel files, Google Forms and the LMS
2. Data Wrangling	Making data clean, correct and usable	Remove duplicates; fix spelling; fill/remove missing values; convert text to numbers; rename columns; merge tables	Replace missing math scores with the column mean and drop duplicate student rows
3. Data Management	How data is stored, organised, updated, protected and access-controlled	Save in folders/databases; proper file naming; backups; keep dataset versions	Store data in `Raw_Data / Clean_Data / Final_Data` folders

A6. A café owner has two years of daily sales data. Explain the four types of data analysis and write one suitable question for each type for this café. **[8]**

MODEL ANSWER

Type	Asks	Café question
Descriptive	What is happening / happened? (past or present)	What were the total sales of each category last month?
Diagnostic	Why is it happening?	Why did drink sales drop in March compared with February?
Predictive	What might happen? (future, based on past patterns)	How many cakes are likely to be sold in December?
Prescriptive	What should we do? (action)	Should we spend 50% of the promotion budget on drinks in December and only 5% in February?

A7. Explain data storytelling and its two essential components. **[4]**

MODEL ANSWER

Data storytelling is the art of translating complex analysis into an easy-to-understand story that engages the audience and makes data relatable.

- **Data visualisation** – presenting data in graphical format (e.g. bar chart of 5-star vs 1-star review counts instead of a raw table) so information is shown concisely.
- **Data narrative** – the written/spoken explanation that tells the audience what the chart means and why it matters (e.g. "142 of 177 reviews are 5-star, so customers are highly satisfied").

A8. Explain the STAR framework and apply it to a data cleaning task. **[5]**

MODEL ANSWER

STAR (Situation – Task – Action – Result) is a structured way to explain a problem/project clearly and logically.

- **Situation**: The student dataset was messy and inconsistent (missing marks, duplicate rows).
- **Task**: Prepare the data for analysis.
- **Action**: Filled missing values with the mean, fixed data formats, removed duplicates.
- **Result**: Data became clean and ready for analysis/modelling.

A related data science workflow: Collect → Clean → Analyse → Model → Visualise → Decide.

SECTION B — Chapter 2: Python Programming

B1. Give four reasons Python is widely used in data analysis and four tasks it can perform in this course. Then differentiate Jupyter Notebook/Google Colab and an IDE. **[4 + 4 = 8]**

MODEL ANSWER

Reasons: easy to learn and read; powerful and flexible; supported by many libraries (NumPy, pandas, Matplotlib); widely used in industry.

Tasks: load datasets, clean data, transform data, analyse data, visualise data, automate analysis tasks.

Jupyter/Colab: runs in a browser, code organised in cells, output appears directly below each cell → suited to data analysis and experimentation (write → run → observe → modify → run again).

IDE (VS Code, PyCharm): full development environment for writing whole programs/projects in files; output appears in a terminal.

B2. State four Python syntax rules. Then identify the error type in each code below and correct it. **[4 + 6 = 10]**

```
(i)  if mark >= 50
         print("Pass")

(ii) student_name = "Ali"
     print(Student_name)

(iii) if mark >= 50:
      print("Pass")
```

MODEL ANSWER

Rules: Python is case-sensitive; indentation defines code blocks (no `{ }`); statements do not need `;`; variable names start with a letter or `_`, no spaces (use `_`); `#` starts a comment.

	Error	Reason	Fix
(i)	SyntaxError	Missing colon after the condition	`if mark >= 50:`
(ii)	NameError	`Student_name` ≠ `student_name` (case-sensitive)	`print(student_name)`
(iii)	IndentationError	Body of `if` not indented	Indent `print("Pass")` by 4 spaces

Debug steps: read the last line of the error → check spelling → brackets → quotation marks → indentation.

B3. Write the output of the following code. **[4]**

```
print(type(20))
print(type(3.25))
print(type("Ali"))
print(type(True))
```

MODEL ANSWER

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

B4. Compare list, tuple, set and dictionary, giving an example of each. **[8]**

MODEL ANSWER

Collection	Property	Example
List	Ordered, changeable, allows duplicates	`shopping = ["Bread", "Milk", "Eggs"]`
Tuple	Ordered, cannot be changed	`point = (3, 5)`
Set	Stores unique values only (no duplicates)	`programs = {"CS", "IT"}`
Dictionary	Key-value pairs, access by key	`student = {"name": "Ali", "programme": "IT", "mark": 68}`

B5. Trace the output. **[6]**

```
shopping = ["Bread", "Milk", "Eggs"]
shopping.append("Rice")
shopping[1] = "Tea"
print(shopping)
print(len(shopping), shopping[-1])
print(shopping[1:3])

student = {"name": "Ali", "mark": 68}
student["mark"] = student["mark"] + 10
student["grade"] = "B"
print(student)
print(len(student))
```

MODEL ANSWER

```
['Bread', 'Tea', 'Eggs', 'Rice']
4 Rice
['Tea', 'Eggs']
{'name': 'Ali', 'mark': 78, 'grade': 'B'}
3
```

Key points: index starts at 0; `-1` = last item; slice `[1:3]` includes index 1, excludes 3; assigning a new key adds it to the dict.

B6. Write a program that stores a mark and prints the grade: 80 and above "Excellent", 60–79 "Good", 50–59 "Pass", below 50 "Fail". **[5]**

MODEL ANSWER

```
mark = 85

if mark >= 80:
    print("Excellent")
elif mark >= 60:
    print("Good")
elif mark >= 50:
    print("Pass")
else:
    print("Fail")
```

Order matters: check the highest range first, because the first true condition wins.

B7. (a) Differentiate a `for` loop and a `while` loop. (b) Trace the output of both programs. (c) What happens if line `n = n // 2` is removed? **[2 + 4 + 2 = 8]**

```
# Program 1
total = 0
for i in range(1, 6):
    if i % 2 == 0:
        continue
    total = total + i
print(total)

# Program 2
n = 10
count = 0
while n > 1:
    n = n // 2
    count = count + 1
print(n, count)
```

MODEL ANSWER

(a) **for**: iterates through a collection/range a known number of times. **while**: repeats as long as a condition is True; used when repetition depends on a condition.
(b) Program 1 adds only odd numbers 1+3+5. Program 2: 10→5→2→1 (3 iterations).

```
9
1 3
```

(c) `n` never changes, the condition `n > 1` stays True forever → **infinite loop**. The condition must eventually become False.

B8. Write a program that stores 5 marks in a list, uses a `for` loop to display each mark with "Pass"/"Fail", counts how many passed and prints the average. **[8]**

MODEL ANSWER

```
marks = [70, 45, 60, 90, 38]
pass_count = 0

for m in marks:
```

```
    if m >= 50:
        print(m, "Pass")
        pass_count = pass_count + 1
    else:
        print(m, "Fail")

average = sum(marks) / len(marks)
print("Number passed:", pass_count)
print("Average:", average)
```

```
70 Pass
45 Fail
60 Pass
90 Pass
38 Fail
Number passed: 3
Average: 60.6
```

B9. Write a simple calculator that uses two numbers and prints addition, subtraction, multiplication and division. The program must not crash when the second number is 0. **[5]**

MODEL ANSWER

```
num1 = 10
num2 = 5

print("Addition:", num1 + num2)
print("Subtraction:", num1 - num2)
print("Multiplication:", num1 * num2)

if num2 != 0:
    print("Division:", num1 / num2)    # 2.0  (/ always returns float)
else:
    print("Division: cannot divide by zero")
```

B10. Define class, object, attribute, method and `self`, using one example. **[5]**

MODEL ANSWER

- **Class** – a blueprint/template for creating objects, e.g. `Student`.
- **Object** – an instance created from a class, e.g. `student1 = Student("Ali", 68)`.
- **Attribute** – data belonging to an object, e.g. `self.name`, `self.mark`.
- **Method** – a function defined inside a class, e.g. `show_info()`, `is_pass()`.
- **self** – refers to the current object, so a method can access that object's own attributes.

`__init__` is the constructor: it runs automatically when an object is created and sets its attributes.

B11. Write a class `Product` with attributes name, price and quantity, a method `total()` that returns price × quantity, and a method `show_info()`. Create two objects and call the methods. Also state why defining the class alone produces no output. **[8]**

MODEL ANSWER

```
class Product:
    def __init__(self, name, price, quantity):
```

```
        self.name = name
        self.price = price
        self.quantity = quantity

    def total(self):
        return self.price * self.quantity

    def show_info(self):
        print("Product:", self.name)
        print("Total:", self.total())

p1 = Product("Latte", 8.5, 2)
p2 = Product("Cake", 6.0, 3)
p1.show_info()
p2.show_info()
```

```
Product: Latte
Total: 17.0
Product: Cake
Total: 18.0
```

Defining a class only creates the blueprint. Output appears only after an object is created and a method is called.

SECTION C — Chapter 3: Python Libraries for Data Analysis

C1. What is a Python library? Explain three limitations of plain Python that make data analysis libraries necessary. **[5]**

MODEL ANSWER

A library is a collection of ready-made code that saves time, reduces code and makes complex tasks easy (one function instead of 100 lines).
Plain Python: (1) is **slow for large data**; (2) has **no built-in table processing**; (3) has **no built-in chart system**. Libraries let us process large datasets, handle tables, compute fast and create charts/reports.

C2. Complete the table for NumPy, Pandas and Matplotlib. **[9]**

MODEL ANSWER

Library	Purpose	Main structure / feature	Import
NumPy (Numerical Python)	Fast mathematical computation, arrays and matrices, scientific computing	`ndarray`; written in C → much faster than lists	`import numpy as np`
Pandas	Data tables, cleaning, transformation, analysis	`DataFrame` (rows + columns, like an Excel/database table)	`import pandas as pd`
Matplotlib	Drawing charts, visualising data, creating graphs	Line, bar, histogram, pie, scatter	`import matplotlib.pyplot as plt`

Later: Scikit-learn → machine learning.

C3. Explain how the three libraries work together in a data analysis workflow, and write a short pipeline that loads `student.csv`, computes the average math score with NumPy and draws a bar chart of average math score by program. **[8]**

MODEL ANSWER

Pandas → load & clean data; NumPy → compute numbers; Matplotlib → draw charts. Together they form a complete analysis system.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("student.csv")                # Pandas: load
df = df.drop_duplicates()                      # Pandas: clean

math = df["math_score"].to_numpy()             # NumPy: compute
print("Average math:", np.mean(math))

avg = df.groupby("program")["math_score"].mean()
plt.bar(avg.index, avg.values)                 # Matplotlib: chart
plt.title("Average Math Score by Program")
```

```
plt.xlabel("Program")
plt.ylabel("Average score")
plt.show()
```

C4. State the most suitable chart for each situation. **[5]**
(i) trend of monthly sales over a year (ii) compare total sales between categories (iii) distribution of exam marks (iv) share of each payment method (v) relationship between screen time and data usage

MODEL ANSWER
(i) line chart `plt.plot()` (ii) bar chart `plt.bar()` (iii) histogram `plt.hist()` (iv) pie chart `plt.pie()` (v) scatter plot `plt.scatter()`

C5. List four common beginner mistakes when using libraries and how to fix each. **[4]**

MODEL ANSWER

Mistake	Fix
Forget to install library (`ModuleNotFoundError`)	`!pip install pandas` in Colab
Wrong import name, e.g. `import panda`	`import pandas as pd`
File not found	Upload file to Colab / check path and spelling
Wrong column name (`KeyError`)	Check `df.columns`; names are case-sensitive

SECTION D — Chapter 5: NumPy & Data Transformation

D1. Explain three limitations of Python lists and how NumPy overcomes them. Illustrate with `[1,2,3] + [4,5,6]`. **[6]**

MODEL ANSWER

Python list	NumPy array
Slower for large data	Faster (implemented in C)
Flexible types, memory-inefficient	Fixed data type, less memory
No vectorised operations – manual loops needed	Vectorised: operation applies to every element at once

```
[1, 2, 3] + [4, 5, 6]                       # list → [1, 2, 3, 4, 5, 6]  (joins)
np.array([1, 2, 3]) + np.array([4, 5, 6])   # array → [5 7 9]           (adds element-wise)
```

NumPy is also the foundation for Pandas, Scikit-learn and TensorFlow, and supports linear algebra.

D2. Write the output. **[5]**

```
import numpy as np
print(np.zeros(3))
print(np.ones(4))
print(np.arange(6))
print(np.arange(2, 11, 3))
print(np.linspace(0, 10, 5))
```

MODEL ANSWER

```
[0. 0. 0.]
[1. 1. 1. 1.]
[0 1 2 3 4 5]
[2 5 8]
[ 0.   2.5  5.   7.5 10. ]
```

`arange(start, stop, step)` excludes stop; `linspace(start, stop, num)` includes stop and gives *num* evenly spaced values. zeros/ones give floats.

D3. Given `a = np.array([[1, 2, 3], [4, 5, 6]])`, write the output of each expression. **[12]**

MODEL ANSWER

Expression	Output	Meaning
`a.shape`	`(2, 3)`	2 rows, 3 columns
`a.ndim`	`2`	number of dimensions
`a.size`	`6`	total elements
`a.dtype`	`int64`	data type of values
`a[1, 2]`	`6`	row 1, column 2
`a[0]`	`[1 2 3]`	first row
`a[:, 1]`	`[2 5]`	column 1 of all rows
`a * 2`	`[[ 2 4 6] [ 8 10 12]]`	vectorised
`a.reshape(3, 2)`	`[[1 2] [3 4] [5 6]]`	3×2, size must stay 6
`np.sum(a)` / `np.mean(a)`	`21` / `3.5`	aggregation
`np.max(a)` / `np.min(a)`	`6` / `1`	aggregation
`a.sum(axis=0)` / `a.sum(axis=1)`	`[5 7 9]` / `[ 6 15]`	per column / per row

`a.reshape(4, 2)` would give a **ValueError** because 4×2 = 8 ≠ 6 elements.

D4. Given `v = np.arange(1, 13)`, write the output of `v[2:7]`, `v[-3:]`, `v[::4]` and `v.reshape(3, 4)[1:, 2:]`. **[4]**

MODEL ANSWER

```
[3 4 5 6 7]
[10 11 12]
[1 5 9]
[[ 7  8]
 [11 12]]
```

D5. (Lab-style) Marks of 8 students are 45, 78, 62, 90, 38, 55, 81, 70. Using NumPy only, write code to: (a) create the array; (b) print mean, median, max, min and standard deviation; (c) count students who passed (≥ 50) and display their marks; (d) label each student Pass/Fail; (e) add 5 bonus marks without exceeding 100. Show the output. **[12]**

MODEL ANSWER

```
import numpy as np

marks = np.array([45, 78, 62, 90, 38, 55, 81, 70])       # (a)

print("Mean:", np.mean(marks))                            # (b)
print("Median:", np.median(marks))
print("Max:", np.max(marks), "Min:", np.min(marks))
print("Std:", round(np.std(marks), 2))

passed = marks >= 50                                      # (c) boolean condition
print("Passed count:", np.sum(passed))
```

```
print("Passed marks:", marks[passed])

print(np.where(marks >= 50, "Pass", "Fail"))             # (d)

print(np.minimum(marks + 5, 100))                        # (e)
```

```
Mean: 64.875
Median: 66.0
Max: 90 Min: 38
Std: 17.0
Passed count: 6
Passed marks: [78 62 90 55 81 70]
['Fail' 'Pass' 'Pass' 'Pass' 'Fail' 'Pass' 'Pass' 'Pass']
[50 83 67 95 43 60 86 75]
```

Note: `np.std` is the population SD (ddof=0); pandas `.std()` uses ddof=1 so it gives a slightly larger value. `marks >= 50` returns a True/False array; summing it counts the Trues.

D6. Differentiate `np.random.rand()` and `np.random.randint()` and give two use cases. Also explain what reshaping is used for. **[5]**

MODEL ANSWER

`np.random.rand(3)` → random **floats between 0 and 1** (e.g. 0.42). `np.random.randint(1, 10, size=5)` → random **integers in a range** (low included, high excluded). Use cases: simulation, testing (e.g. generate dummy marks).
Reshaping changes the array structure without changing data: 1D → 2D for analysis (e.g. turn a time series into a matrix), or flatten 2D → 1D to feed image data into a machine-learning model.

D7. Define data transformation and explain why it is important. Give four examples of transformation. **[6]**

MODEL ANSWER

Data transformation = changing data into a suitable format for analysis.
Why: raw data is rarely ready for analysis; improves data usability; improves (future) model performance; makes visualisation easier.
Examples: create new columns, convert formats, normalise values, group data (also rename, encode, bin).

D8. Explain the stages of the data pipeline and name one pandas function used at each stage. **[7]**

MODEL ANSWER

Stage	Meaning	pandas example
Load	Bring data from outside into the program	`pd.read_csv()`
Inspect	Quick scan, like opening a file	`head(), tail(), shape, info()`
Profile	Understand behaviour: mean, distribution, unique values, outliers	`describe(), value_counts(), nunique()`
Clean	Remove/handle missing values, fix formats, check outliers, remove duplicates	`fillna(), drop_duplicates(), astype()`
Validate	Quality control – ensure data makes sense	`isnull().sum()`, check scores between 0–100
Transform	Reshape into analysis-ready form	new column, `rename(), pd.cut(), get_dummies()`
Analyse	Extract knowledge: statistics, trends, models	`groupby().mean()`, charts

D9. Define feature engineering and give three examples from different domains. **[4]**

MODEL ANSWER

Feature engineering = creating new useful variables from existing data.
Height + Weight → BMI (health); Price × Quantity → Total Sales (sales analytics); years worked → experience level (HR); marks → grade category (education).

D10. (Coding – very likely) The DataFrame `df` is given:

```
import pandas as pd
df = pd.DataFrame({"Name": ["Alice","Bob","Charlie","David","Ema"],
                   "Age": [22, 35, 28, 45, 19],
                   "Price": [10.0, 7.5, 5.0, 12.0, 8.0],
                   "Quantity": [2, 3, 4, 1, 5]})
```

Write ONE pandas statement for each task. **[20]**

MODEL ANSWER

Task	Code
(a) Create column Total = Price × Quantity	`df["Total"] = df["Price"] * df["Quantity"]`
(b) Rename Quantity to Qty	`df = df.rename(columns={"Quantity": "Qty"})`
(c) Select only Name and Total	`df[["Name", "Total"]]`
(d) Drop column Age	`df.drop(columns=["Age"])`
(e) Filter rows with Total > 20	`df[df["Total"] > 20]`
(f) Filter Age < 30 AND Total ≥ 20	`df[(df["Age"] < 30) & (df["Total"] >= 20)]`
(g) Sort by Total, highest first	`df.sort_values("Total", ascending=False)`
(h) Convert names to uppercase using apply	`df["Name"] = df["Name"].apply(lambda x: x.upper())`
(i) Category: Total ≥ 20 → "High" else "Low"	`df["Level"] = df["Total"].apply(lambda x: "High" if x >= 20 else "Low")`
(j) Bin Age into Young (≤25), Adult (26–40), Senior (>40)	`df["Age_Group"] = pd.cut(df["Age"], bins=[0,25,40,100], labels=["Young","Adult","Senior"])`
(k) Min-max normalise Price to 0–1	`df["Price_norm"] = (df["Price"] - df["Price"].min()) / (df["Price"].max() - df["Price"].min())`
(l) Average Total per Age_Group	`df.groupby("Age_Group")["Total"].mean()`
(m) Count, sum and mean of Total per group	`df.groupby("Age_Group")["Total"].agg(["count","sum","mean"])`
(n) One-hot encode Age_Group	`pd.get_dummies(df["Age_Group"])`
(o) Label-encode a Gender column (Male→0, Female→1)	`df["Gender_code"] = df["Gender"].map({"Male": 0, "Female": 1})`
(p) Combine First + Last into FullName	`df["FullName"] = df["First"] + " " + df["Last"]`
(q) Convert text date to date format	`df["date"] = pd.to_datetime(df["date"])`
(r) Convert number stored as text to numeric	`df["Price"] = pd.to_numeric(df["Price"])` or `.astype(float)`

(s) Extract day name from date	`df["Day"] = df["date"].dt.day_name()`
(t) Top 3 by Total	`df.nlargest(3, "Total")`

Expected results after (a), (e), (j), (k), (m):

```
      Name  Age  Price  Quantity  Total
0    Alice   22   10.0         2   20.0
1      Bob   35    7.5         3   22.5
2  Charlie   28    5.0         4   20.0
3    David   45   12.0         1   12.0
4      Ema   19    8.0         5   40.0

(e) Total > 20  → Bob (22.5), Ema (40.0)
(j) Age_Group   → Alice Young, Bob Adult, Charlie Adult, David Senior, Ema Young
(k) Price_norm  → 0.714, 0.357, 0.000, 1.000, 0.429

(m)        count   sum   mean
Young          2  60.0  30.00
Adult          2  42.5  21.25
Senior         1  12.0  12.00
```

Common traps: double brackets `[[ ]]` to select many columns; use `&` / `|` with brackets around each condition (not `and` / `or`); `rename` / `drop` return a new DataFrame unless you assign it back.

D11. Differentiate the following pairs of terms with an example: (a) grouping vs aggregation (b) categorising vs binning (c) encoding vs normalising. **[6]**

MODEL ANSWER

(a) **Grouping** puts similar rows together by a category (group sales by month); **aggregation** combines values into a summary result (total/average/count of each group). Usually used together: `groupby("month")["sales"].sum()`.
(b) **Categorising** assigns labels using a rule (score ≥ 50 → Pass, else Fail); **binning** groups continuous values into ranges (age 18–25 Young, 26–40 Adult) using `pd.cut()`.
(c) **Encoding** converts text categories to numbers so computers/ML models can use them (Male→0, Female→1, or `get_dummies`); **normalising** rescales numeric values to a similar range, e.g. 0–1, so they can be compared fairly.

D12. State three common mistakes in data transformation and how to avoid them. **[3]**

MODEL ANSWER

(1) **Overwriting original data** → work on a copy `df_clean = df.copy()`. (2) **Wrong data type** (numbers stored as text, cannot calculate) → check `df.dtypes`, convert with `astype` / `pd.to_numeric`. (3) **Incorrect grouping** (wrong column or inconsistent labels "cs" vs "CS") → standardise text first, e.g. `str.upper()`, then group.

SECTION E — Lab-Based Case Study (Pandas & Data Preparation)

This section mirrors Lab Assessment 1. Expect one long structured question built on one of the five lab datasets (student, online shopping, café sales, gym, mobile usage). Practise the code until you can write it without notes.

E1. You are a junior data analyst given `student.csv` (columns: student_id, program, gender, math_score, programming_score, data_mining_score). Write pandas code to complete the tasks. **[20]**

a. Import pandas and load the file into `df`.
b. Display the first 5 and last 5 rows, the shape, column names and data types.
c. Show the number of missing values per column and replace numeric missing values with the column mean.
d. Check for duplicate rows and remove them.
e. Ensure the score columns are numeric.
f. Produce descriptive statistics.
g. Create `average_score` and find the mean average_score per program and per gender.
h. Display the top 5 and bottom 5 students by average_score.

MODEL ANSWER

```
# (a) Setup & loading
import pandas as pd
import numpy as np
df = pd.read_csv("student.csv")

# (b) Inspect
print(df.head())          # first 5 rows
print(df.tail())          # last 5 rows
print(df.shape)           # (rows, columns)
print(df.columns)         # column names
print(df.dtypes)          # data types   (or df.info())

# (c) Missing values
print(df.isnull().sum())
num_cols = ["math_score", "programming_score", "data_mining_score"]
for col in num_cols:
    df[col] = df[col].fillna(df[col].mean())

# (d) Duplicates
print("Duplicates:", df.duplicated().sum())
df = df.drop_duplicates()

# (e) Correct data types
df[num_cols] = df[num_cols].apply(pd.to_numeric, errors="coerce")
print(df.dtypes)

# (f) Descriptive statistics
print(df.describe())

# (g) New column + group-based analysis
```

```
df["average_score"] = df[num_cols].mean(axis=1)
print(df.groupby("program")["average_score"].mean())
print(df.groupby("gender")["average_score"].mean())

# (h) Ranking & filtering
print(df.sort_values("average_score", ascending=False).head(5))   # top 5
print(df.sort_values("average_score").head(5))                    # bottom 5
# alternative: df.nlargest(5, "average_score"), df.nsmallest(5, "average_score")
```

Order tip: remove duplicates *before* filling missing values if you want the mean to be unbiased – duplicate rows count twice in the mean. Either order is accepted if you justify it.

E2. Trace the code below and write the output of each print statement. **[10]**

```
import pandas as pd
import numpy as np
t = pd.DataFrame({"student_id": [1, 2, 3, 3, 4, 5],
                  "program": ["CS", "IT", "CS", "CS", "IT", "CS"],
                  "math_score": [80, np.nan, 70, 70, 90, 60],
                  "programming_score": [75, 85, np.nan, np.nan, 95, 65]})
print(t.isnull().sum())                                    # (i)
print(t.duplicated().sum())                                # (ii)
t = t.drop_duplicates()
print(t.shape)                                             # (iii)
print(t["math_score"].mean())                              # (iv)
t["math_score"] = t["math_score"].fillna(t["math_score"].mean())
t["programming_score"] = t["programming_score"].fillna(t["programming_score"].mean())
print(t.groupby("program")["math_score"].mean())           # (v)
```

MODEL ANSWER

```
(i)   student_id           0
      program              0
      math_score           1
      programming_score    2
      dtype: int64
(ii)  1
(iii) (5, 4)
(iv)  75.0          # (80 + 70 + 90 + 60) / 4, NaN is skipped
(v)   program
      CS    70.0     # (80 + 70 + 60) / 3
      IT    82.5     # (75 + 90) / 2, the filled 75 counts
      Name: math_score, dtype: float64
```

Row 3 and its copy are identical (NaN values are treated as equal by `duplicated()`), so 1 duplicate is removed.

E3. The output of `df.describe()` for the gym dataset is shown. Interpret **two** numeric columns. **[6]**

	age	visits_per_month	monthly_fee
count	200	200	200
mean	23.4	9.8	85.0
std	3.1	6.2	20.5
min	18	0	50
25%	21	4	70
50%	23	8	80
75%	25	15	100
max	35	28	150

MODEL ANSWER

age: 200 members, average age 23.4 years with a small spread (std 3.1). Half are 23 or younger and 75% are 25 or younger, so members are mostly young adults/university students. The max of 35 is a few older members.
visits_per_month: average 9.8 visits but std 6.2 is large relative to the mean → attendance varies a lot. The median (8) is lower than the mean, so a few very active members (max 28) pull the mean up (right-skewed). Min 0 means some members pay but never come – a retention risk the gym could target with reminders.
Method: count → mean → spread (std / min / max) → median vs mean → real-world meaning.

E4. Compare NumPy and pandas, and justify when you would use each in your lab work. **[7]**

MODEL ANSWER

Aspect	NumPy	pandas
Structure	`ndarray` – one data type, no labels	`DataFrame` / `Series` – labelled rows & columns, mixed types
Strength	Fast numerical computation, vectorised maths, boolean masks	Loading CSV, cleaning (missing values, duplicates), groupby, reporting
Missing values	Basic (NaN breaks `np.mean` unless `np.nanmean`)	Built-in `isnull, fillna, dropna`
Text columns	Not suitable	Handles program, gender, etc.

Justification: I used pandas to load `student.csv`, fix missing values and group scores by program, because those tasks need column names and text categories. I converted `math_score` to a NumPy array to calculate mean/std and count students ≥ 50 with a boolean mask, because NumPy is faster and simpler for pure numeric work. pandas is built on NumPy, so they work together.

E5. Reflection questions (structured, 4 parts). **[16]**
(a) Describe data quality issues found and their impact on statistical results. (b) Explain one interesting pattern and how it supports decision making. (c) Discuss two limitations of the dataset/analysis. (d) Propose two further analyses.

MODEL ANSWER

(a) Data quality – Issues: missing values in math_score and programming_score; duplicate student records; score columns stored as text. Impact: duplicates count a student twice and bias the mean towards that student; missing values make `mean()` use fewer records or break calculations; text-type scores cannot be averaged. Cleaning improved reliability: after removing duplicates and filling missing values with the mean, every student is counted once and all rows can be used, so the averages reflect the real class.
(b) Pattern – e.g. IT students' average math score (82.5) is higher than CS students' (70.0). Real-world meaning: CS students may need more maths support. Decision: the faculty could offer extra maths tutorials or early-warning monitoring for CS students.
(c) Limitations – (1) Small sample from one university → cannot generalise to all students (sample vs population). (2) Mean imputation hides true variation and lowers the standard deviation. (3) Missing factors such as attendance, study hours or family background, so we can see *what* differs but not *why* – conclusions are descriptive, not causal.
(d) Further analysis – (1) Collect attendance and study-hours data to do **diagnostic** analysis (correlation between attendance and score). (2) Use several semesters of data to **predict** at-risk students (e.g. regression/classification with Scikit-learn), allowing early intervention. (3) Visualise score distributions per program with histograms/box plots to communicate findings.

E6. Write a mini analysis for the café dataset (columns: date, item, category, quantity, unit_price, total_sales, payment_type): verify total_sales, find total sales per category, the most popular payment type, the best-selling day of the week, and plot sales per category. **[10]**

MODEL ANSWER

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("cafe_sales.csv")
df["date"] = pd.to_datetime(df["date"])                    # convert format

# validate: recompute total and compare
df["check_total"] = df["quantity"] * df["unit_price"]
print("Wrong totals:", (df["check_total"] != df["total_sales"]).sum())

cat_sales = df.groupby("category")["total_sales"].sum().sort_values(ascending=False)
print(cat_sales)

print(df["payment_type"].value_counts())                   # most popular first

df["day"] = df["date"].dt.day_name()                       # feature engineering
print(df.groupby("day")["total_sales"].sum().idxmax())     # best day

cat_sales.plot(kind="bar", title="Total Sales by Category")
plt.ylabel("Sales (RM)")
plt.show()
```

Appendix — One-Page Cheat Sheet

Need	Code
Load / inspect	`pd.read_csv("f.csv")`, `df.head()`, `df.tail()`, `df.shape`, `df.columns`, `df.dtypes`, `df.info()`
Missing values	`df.isnull().sum()`, `df[c].fillna(df[c].mean())`, `df.dropna()`
Duplicates	`df.duplicated().sum()`, `df.drop_duplicates()`
Types	`df[c].astype(int)`, `pd.to_numeric(df[c])`, `pd.to_datetime(df[c])`
Statistics	`df.describe()`, `.mean() .median() .std() .min() .max() .count()`, `value_counts()`
Select / filter / sort	`df[["a","b"]]`, `df[df[c] > 50]`, `df.sort_values(c, ascending=False)`, `nlargest(5, c)`
Transform	`df["new"] = …`, `rename(columns={})`, `drop(columns=[])`, `apply(lambda x: …)`, `map({})`
Bin / encode / normalise	`pd.cut(col, bins, labels)`, `pd.get_dummies(col)`, `(x - min) / (max - min)`
Group	`df.groupby(g)[c].mean()`, `.agg(["count","sum","mean"])`
NumPy create	`np.array, zeros, ones, arange(start,stop,step), linspace(start,stop,num)`
NumPy attributes	`shape, ndim, dtype, size`; `reshape(r, c)`
NumPy maths	`a+b, a*2, np.sqrt(a), np.sum/mean/max/min/std/median`, `a[a>=50]`, `np.where(cond, x, y)`
Random	`np.random.rand(n)` (0–1 floats), `np.random.randint(low, high, size)`
Charts	`plt.plot / bar / hist / pie / scatter`, `plt.title, xlabel, ylabel, show`

Definitions to memorise word-for-word

- **Data preparation**: collecting, transforming and storing a dataset in a form useful for analysis (Collection → Wrangling → Management).
- **Data analysis**: examining prepared data to uncover insights and guide decision making.
- **Data storytelling**: communicating insights through summaries, visualisations and narratives.
- **Metadata**: data about data (variable names, units, creation date, number of missing values).
- **Data transformation**: changing data into a suitable format for analysis.
- **Feature engineering**: creating new useful variables from existing data.
- **Vectorisation**: applying an operation to the whole array at once without a manual loop.
- **Class / object / method / self**: blueprint / instance / function inside class / current object.

Prediction paper prepared for revision. Question style and weighting are predictions based on the lecture notes and Lab 1 marking scheme, not the actual exam.